

EIN092B: Laboratorio de Técnicas de Visualización para Clasificación y Predicción

Guía de Actividades Sesión 13

4 al 7 de noviembre de 2025

1. Requisitos de entrada.

- Haber **entendido** y **realizado** las actividades de todas las sesiones anteriores.
- Haber revisado los conceptos asociados a las técnicas de aprendizaje supervisado estudiadas en el curso previo de ciencia de datos, tales como regresión logística, *K-nearest neighbor*, máquinas de soporte vectorial y regresión lineal. Adicionalmente, haber revisado técnicas de validación cruzada como *k-fold cross validation*.
- Haber estudiado la codificación de variables categóricas en *one-hot vectors*.

2. Objetivos

- Explorar técnicas de visualización para resultados de modelos de clasificación y regresión.
- Experimentar con distintas métricas de evaluación y gráficos para visualizar errores en tareas de clasificación o regresión.

3. Indicaciones generales.

Las actividades planteadas en esta guía deben ser entregadas durante la sesión de laboratorio por lo menos hasta la Sección 5.1.2. La actividad propuesta en la Sección 5.1.3 puede ser entregada sin validación durante la sesión.

3.1. Sobre el proceso de revisión de actividades.

Para el proceso de revisión consideren lo siguiente:

- La revisión y evaluación de cada una de las actividades se realizará en dos etapas: (i) una validación básica de requerimientos funcionales mínimos durante la sesión, y (ii) una verificación de requerimientos funcionales y no funcionales posterior a la sesión.

- Durante la sesión, avise al staff (profesor o ayudante) cuando complete una o varias actividades para que muestre y explique sus resultados, y responda las preguntas que se le realizarán sobre los conceptos asociados.
- Cualquier integrante del grupo debe ser capaz de responder las preguntas. Si las respuestas no son satisfactorias, o algún integrante no es capaz de responder, no se revisará la actividad hasta que demuestren que **TODOS** los integrantes del grupo demuestren un entendimiento mínimo sobre lo que están entregando.
- Durante la sesión, el staff marcará en una planilla cuando Ud. considere una actividad terminada y haya pasado la interrogación. El tener la actividad marcada como revisada en la sesión no entrega puntaje, y solo asegura que cumple con requerimientos para que entregue sus archivos y pase a la siguiente etapa de revisión. El formato de entrega de sus archivos se indica en la Sección 3.2.
- En el caso de las actividades no entregadas en la sección correspondiente, contará como no entregada a tiempo y **deberá entregar la actividad corregida en la siguiente sesión, aplicando descuentos por atraso (penalización correspondiente al 35 % del puntaje inicial si se entrega en la siguiente sesión. Si se entrega en dos sesiones posteriores, 50 %).**
- En el caso de que algún integrante del grupo no este presente cuando su grupo correspondiente presente la evaluación y su correspondiente interrogación, deberá presentarlo por su propia cuenta en las siguientes sesiones y sufriendo la penalización por atraso.

3.2. Indicaciones de formato para entrega.

La evaluación constará de dos partes. La primera parte consiste en una validación básica y preguntas sobre los códigos propuestos por el grupo. La segunda parte constará de una verificación de sus códigos y respuestas, la cual se realizará sobre los archivos que deberá entregar vía Aula USM, los cuales deben seguir estrictamente las especificaciones que se le indican a continuación:

1. Debe crear una carpeta con el nombre *GXX_Sesion13*, donde *XX* el número de grupo. Por ejemplo, la carpeta para el grupo 5 se llamaría *G5_Sesion13*. El número de su grupo fue definido en la sesión anterior.
2. Dentro de esta carpeta, debe incluirse un archivo de Jupyter Notebook que contenga el desarrollo de las actividades y las respuestas correspondientes.

Recordar que **será responsabilidad del grupo entregar de manera correcta los archivos**. Se permitirá el envío de los archivos hasta las **23:59 del 8 de Noviembre**. Sin embargo, solo se evaluarán las actividades marcadas como revisadas durante la sesión. Cualquier actividad adicional no será considerada en la revisión.

4. Técnicas de evaluación de algoritmos de clasificación y regresión

En esta sección se presenta una breve introducción a las técnicas más comunes para evaluar modelos o algoritmos en problemas de regresión y clasificación. Se discutirán métricas y gráficos

fundamentales como la matriz de confusión y la curva ROC para problemas de clasificación, así como métricas para la evaluación de modelos de regresión, proporcionando una base para comparar el desempeño de los modelos estudiados en este laboratorio.

4.1. Problemas de clasificación

En distintos ámbitos científicos e ingenieriles, para evaluar la eficacia de un algoritmo, se suele recurrir a información predefinida que representa el resultado esperado para la resolución de cierto problema y así poder evaluar el rendimiento del algoritmo. Esta información, conocida como *groundtruth*, se utiliza como referencia al comparar el rendimiento de un algoritmo.

El éxito o fallo de un algoritmo suele evaluarse mediante métricas que cuantifican su rendimiento en la detección de una situación o evento, en comparación con el *groundtruth*. Para esto, existen cuatro categorías generales:

- **Verdadero Positivo (TP):** Ocurre cuando el algoritmo detecta una situación real, es decir, algo que está presente tanto en la *groundtruth* como en los resultados del algoritmo.
- **Falso Negativo (FN):** Representa una situación que el algoritmo no detecta un determinado fenómeno o evento. Para un problema de clasificación, es el caso en el que el algoritmo no consigue clasificar correctamente según la etiqueta del *groundtruth*.
- **Falso Positivo (FP):** El algoritmo detecta una situación irreal (existe sólo en los resultados del algoritmo).
- **Verdadero Negativo (TN):** Elemento que no aparece ni en el *groundtruth* ni en los resultados del algoritmo.

Una matriz de confusión, es una forma conveniente de mostrar y visualizar la información contenida en estas categorías. Se puede determinar fácilmente cuales observaciones fueron etiquetadas incorrectamente y cuales correctamente. Además, teniendo el número total de observaciones, se pueden obtener estadísticas y la tasa de error global.

A partir de TN, TP, FN, FP se obtienen otras métricas¹:

- **Precisión:** Mide la proporción de verdaderos positivos sobre todos los casos positivos detectados por el algoritmo:

$$\text{Precisión} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (1)$$

Esta métrica es esencial para entender la capacidad del modelo para evitar identificar erróneamente una señal negativa como positiva.

- **Sensibilidad:** Se define como la proporción de verdaderos positivos sobre todas las situaciones reales:

$$\text{Sensibilidad} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (2)$$

Esta métrica refleja la capacidad del modelo para identificar todas las señales positivas en el conjunto de datos.

¹Existen otras métricas como F1-score que corresponde a la media armónica entre la precisión y la sensibilidad.

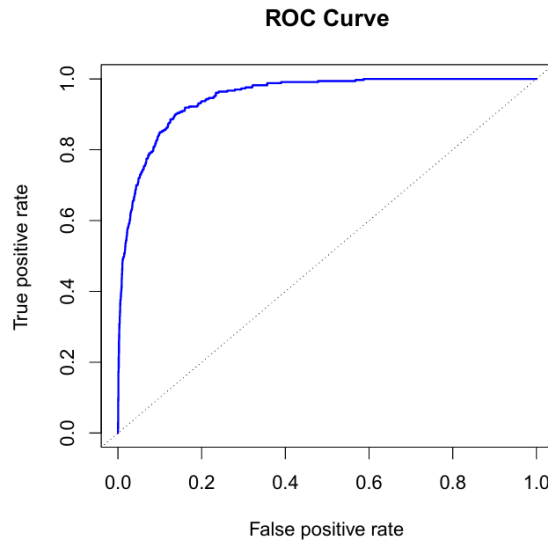


Figura 1: Ejemplo de curva ROC. Esta curva muestra cómo varían dos tipos de error al ajustar el umbral de clasificación. La tasa de verdaderos positivos (*sensibilidad*) representa la fracción de casos correctamente identificados con un umbral dado, mientras que la tasa de falsos positivos ($1 - \textit{especificidad}$) indica la fracción de casos negativos clasificados incorrectamente como positivos con el mismo umbral.

- **Especificidad:** Mide la proporción de verdaderos negativos sobre todos los casos negativos:

$$\text{Especificidad} = \frac{\text{TN}}{\text{FP} + \text{TN}} \quad (3)$$

Esta métrica cuantifica el rendimiento del algoritmo para evitar detectar erróneamente situaciones irrelevantes.

- **Exactitud o *accuracy*:** Mide el número total de predicciones correctas sobre el total de predicciones realizadas:

$$\text{Exactitud} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}} \quad (4)$$

Cada una de estas métricas ofrece una perspectiva diferente sobre la capacidad de un algoritmo para realizar predicciones precisas y útiles, proporcionando así una base sólida para evaluarlo frente al *groundtruth*.

4.2. Curva ROC

La curva ROC es una herramienta utilizada principalmente para evaluar el rendimiento de modelos de clasificación binaria. La curva ROC es un gráfico popular para mostrar simultáneamente dos tipos de errores para todos los umbrales posibles. El nombre ROC es histórico y procede de la teoría de las comunicaciones. Es un acrónimo de *receiver operating characteristics* (características operativas del receptor) [2].

La Figura 1 muestra la curva ROC de un clasificador en los datos de entrenamiento. Los dos errores que se pueden visualizar son la *sensibilidad* y uno menos la *especificidad*. El rendimiento global de un clasificador, sumado sobre todos los umbrales posibles, viene dado por el *área bajo la curva* (AUC). En el caso de un clasificador ideal, una curva ROC deberá estar cerca de la esquina superior izquierda, por lo que cuanto mayor sea el área bajo la AUC, mejor será el clasificador. Para los datos en la Figura 1, el AUC es de 0.95, que se acerca al máximo de 1.0, por lo que se consideraría muy bueno. Se espera que un clasificador cuyo rendimiento sea mejor que el azar tenga un AUC de 0,5 (cuando se evalúa en un conjunto de pruebas independiente no utilizado en el entrenamiento del modelo).

Para un modelo que produce probabilidades para cada clase, permite ajustar un umbral de decisión y, al variar el umbral del clasificador, cambia su tasa de verdaderos positivos (TPR) y falsos positivos (TNR). Éstas también se denominan *sensibilidad* y uno menos la *especificidad* de nuestro clasificador. La curva ROC ideal abraza la esquina superior izquierda, lo que indica una alta tasa de verdaderos positivos y una baja tasa de falsos positivos.

Las curvas ROC son útiles para comparar distintos clasificadores, ya que tienen en cuenta todos los umbrales posibles. El uso de la curva ROC tiene algunas restricciones. Se requieren las probabilidades de que cada muestra pertenezca a la clase positiva. Esto es aplicable a algoritmos de redes neuronales artificiales, regresión logística, entre otros.

4.3. Problemas de regresión

Para evaluar algoritmos para resolver problemas de regresión, se utilizan métricas de error como *mean absolute error* (MAE), *mean squared error* (MSE), *root mean squared error* (RMSE) o el coeficiente de determinación (R^2) para evaluar el rendimiento. Comúnmente, estas métricas se representan en *box plot* o *scatter plot*, y cuando se obtienen múltiples valores para las métricas (como en *cross-validation*) es recomendable incluir una barra con la desviación estándar para hacer referencia a la variación del error.

Para estos casos, gráficos como la curva ROC no es aplicable directamente a problemas de regresión, debido a que estos no producen valores de decisión binarios ni probabilidades. Un problema de regresión puede ser transformado en un problema de clasificación binaria (por ejemplo, seleccionando un umbral de decisión para categorizar un valor continuo en clase 0 o 1), entonces se podría una curva ROC en ese contexto.

5. Actividades

En las siguientes subsecciones se describen las actividades a realizar. En cada actividad encontrará indicaciones generales sobre el conjunto de datos a emplear e indicaciones sobre los requerimientos. En este laboratorio, se emplearán técnicas clásicas para evaluar algoritmos al resolver problemas de clasificación o regresión.

5.1. Aplicación de aprendizaje automático

En esta primera sección, se emplea el dataset público de enfermedades cardíacas [1]. El propósito es aplicar técnicas basadas en datos para abordar un problema relacionado con la detección de enfermedades cardíacas, donde el objetivo final es integrar los algoritmos en una solución práctica. *Heart disease dataset* está disponible en *Kaggle*. En el mismo lugar, puede acceder a una descripción del dataset.

Para cargar los datos en un DataFrame de Pandas y mantener las variables categóricas en su forma original, se provee el siguiente código de referencia. Se busca que estos cambios permitan conservar la representación original de las variables categóricas, lo que facilite los análisis e interpretaciones posteriores.

```

1 import numpy as np
2 import pandas as pd
3 headers = ['age', 'sex', 'chest_pain', 'blood_p', 'serum', 'blood_s', 'electro', '
    max_heart', 'angina', 'oldpeak', 'slope', 'vessel', 'thal', 'normal']
4 df = pd.read_csv("heart.dat", header=None, names=headers, sep=' ')
5 df['sex'][df['sex'] == 0] = 'female'
6 df['sex'][df['sex'] == 1] = 'male'
7 df['chest_pain'][df['chest_pain'] == 1] = 'typical angina'
8 df['chest_pain'][df['chest_pain'] == 2] = 'atypical angina'
9 df['chest_pain'][df['chest_pain'] == 3] = 'non-anginal pain'
10 df['chest_pain'][df['chest_pain'] == 4] = 'asymptomatic'
11 df['blood_s'][df['blood_s'] == 0] = 'lower than 120mg/ml'
12 df['blood_s'][df['blood_s'] == 1] = 'greater than 120mg/ml'
13 df['electro'][df['electro'] == 0] = 'normal'
14 df['electro'][df['electro'] == 1] = 'ST-T wave abnormality'
15 df['electro'][df['electro'] == 2] = 'left ventricular hypertrophy'
16 df['angina'][df['angina'] == 0] = 'no'
17 df['angina'][df['angina'] == 1] = 'yes'
18 df['slope'][df['slope'] == 1] = 'upsloping'
19 df['slope'][df['slope'] == 2] = 'flat'
20 df['slope'][df['slope'] == 3] = 'downsloping'
21 df['thal'][df['thal'] == 3] = 'normal'
22 df['thal'][df['thal'] == 6] = 'fixed defect'
23 df['thal'][df['thal'] == 7] = 'reversible defect'

```

Indicaciones para cargar y preprocesar los datos:

- El conjunto de datos contiene variables categóricas, y estas pueden usarse para alimentar un modelo de aprendizaje automático si se emplea una representación adecuada de los datos. Codifique las variables categóricas como *one-hot vectors*, indicando con un 1 la presencia del atributo en cuestión.
 - Por ejemplo, si un dato recolectado para un paciente incluye el atributo 'angina: yes', este se codificará como [0,1], mientras que si incluye el atributo 'angina: no', se codificará como [1,0]. ¿Por qué es importante representar este tipo de variables con una codificación *one-hot*?

```

1 df = pd.get_dummies(df)
2 df.head()

```

5.1.1. Predicción de la presión sanguínea (40 puntos)

El objetivo de esta sección es predecir el comportamiento de la variable de presión sanguínea en función de las demás variables, para luego cuantificar la diferencia entre el valor real y el valor predicho.

Actividades a realizar:

- Visualice e inspeccione el conjunto de datos para dimensionar el problema e identificar tendencias o comportamientos en las variables. Realice gráficos o proporcione análisis sobre el conjunto de datos. Para los gráficos, realice solo los que considere más importantes y apropiados (por ejemplo, histogramas, boxplots, scatter plots, análisis de covarianza, correlación, etc.). ¿Se observan diferencias entre los pacientes sanos y enfermos?

- Como referencia, se tiene que altos valores asociados a la presión sanguínea (*blood pressure*) pueden ser un factor o indicador de problemas cardíacos. Es decir, un comportamiento anormal de la variable presión sanguínea es un buen indicador para la detección de enfermedades. **El objetivo de esta sección es predecir el comportamiento de la variable de presión sanguínea en función de las demás variables, para luego cuantificar la diferencia entre el valor real y el valor predicho.**

- Extraiga la información de la enfermedad cardíaca (*clase binaria*), además de la variable continua que nos interesará predecir en esta instancia (*target*) con un modelo de regresión lineal.

```
1 class_label = df.pop("normal").values -1 # 0 significa ausencia, 1
  significa presencia
2 reg_label = df.pop("blood_p").values
3 X_data = df.values
4
```

- Divida el conjunto de datos en un set para entrenamiento y otro para test. Extraiga el 30 % para el conjunto de test. Seguidamente, realice una estandarización de los datos (restar la media y dividir por la desviación estándar).
- **Seleccione un modelo para resolver el problema de regresión.** Considere que su elección debe alinearse con su conocimiento previo sobre el modelo. De esta forma será más fácil realizar las actividades y responder las preguntas sobre la actividad. En este caso, se requiere estimar la presión sanguínea a partir de las señales de entrada.²
 - Ajuste los parámetros del modelo³ y luego, mida los errores de predicción para cada dato en el conjunto de entrenamiento. A fin de evaluar las predicciones del modelo de regresión lineal, utilice el MAE y el MSE. Discuta los resultados en el conjunto de entrenamiento y testeo.

²Existen técnicas para determinar que variables independientes influyen más en el proceso de estimar la variable dependiente. Por ejemplo, en regresión lineal, comúnmente se emplean métodos iterativos de selección de variables como *forward step-wise selection*. Por otra parte, modelos como *random forest* y *decision trees* permiten acceder a la importancia de cada característica (*feature important*)

³Existen múltiples formas de seleccionar los hiperparámetros de su modelo, una forma clásica es mediante prueba y error para un conjunto representativo de distintas combinaciones. Otra opción, es usar métodos como *GridSearchCV*.

```
1 from sklearn.metrics import mean_absolute_error, mean_squared_error
2 print("MSE train: ", mean_squared_error(REFERENCIA, PREDICCIÓN))
3 print("MSE test: ", mean_squared_error(REFERENCIA, PREDICCIÓN))
4
```

Si utiliza el método de prueba y error para ajustar los hiperparámetros, asegúrese de probar al menos 5 combinaciones distintas. En el caso de usar *GridSearchCV* para seleccionar la combinación de hiperparámetros, use CV con el mismo número de K . Realice una tabla donde reporte las combinaciones de hiperparámetros y los correspondientes MAE y MSE para cada subconjunto de datos.

- Evalué la calidad del modelo usando *cross-validation* con un número de *fold* igual a $K = 5$. Para cada *fold* se deben reajustar los pesos del modelo. Compare esta estimación en el conjunto de pruebas (test).
- Una vez obtenidos los valores de MAE and MSE para cada una de las combinaciones. Realice un gráfico de *boxplot* para visualizar los valores de MAE y MSE en cada *fold* a fin de representar la variabilidad en las métricas de error. Esto permite observar cómo fluctúan los errores en diferentes particiones del conjunto de datos.
- Por último, reporte el MSE y MAE para las cinco iteraciones como $\text{MEAN} \pm \text{STD}$, donde STD y MEAN corresponden a la media y la desviación estándar de los cinco valores de MAE y MSE. Discuta los resultados.

5.1.2. Predicción de la presión sanguínea (40 puntos)

En esta sección nos enfocaremos en la predicción de la enfermedad cardíaca, aprovechando los experimentos y procesamiento realizados en la actividad 5.1.1. La predicción de la enfermedad cardíaca corresponde a un problema de clasificación binario. En este caso, exploraremos la hipótesis de que el comportamiento anormal de presión sanguínea sobre las personas es un indicio de alguna enfermedad.

Actividades a realizar:

- Se requiere clasificar si un paciente está enfermo o no; para esto se pide que emplee algoritmos de clasificación binaria. Específicamente, *K-nearest neighbor* (KNN) y la *logistic regression* (LR), los cuales se entrenarán para aprender directamente la tarea de manera supervisada. Reporte los resultados.
- Con el fin de obtener una explicación visual a la clasificación de ambos modelos, proyecte los datos en 2 ó 3 dimensiones. Utilice la técnica de reducción de dimensionalidad PCA para representar los datos en las dimensiones deseadas. Comente sobre el comportamiento. Puede asignar colores distintos a los datos según el estado del paciente (enfermedad) y/o la predicción de alguno de los modelos.
- Realice el flujo de entrenamiento y validación de los modelos seleccionados, pero esta vez considerando para el entrenamiento una representación reducida generada por PCA.

- ¿Qué tanto se ve afectada la calidad de la predicción al utilizar la representación reducida generada por PCA en comparación con la representación original? ¿Es esperable este fenómeno? ¿Podría mejorar la calidad de la predicción?
- Finalmente, haga un gráfico de la curva ROC y una matriz de confusión. Si bien KNN no cumple con los requisitos para realizar una curva ROC, la librería de *scikit-learn* permite obtener una probabilidad en base a la medida de distancia de los vecinos si se usa el método `predict_proba`. Discuta los resultados.

5.1.3. Explicabilidad de los modelos (20 puntos)

En esta actividad, se busca analizar y explicar el comportamiento de los modelos de clasificación entrenados previamente. Para ello, se utilizarán técnicas de interpretabilidad basadas en SHAP (SHapley Additive exPlanations), que permiten comprender la contribución de cada característica en las predicciones.

El objetivo es identificar qué variables tienen mayor impacto en la decisión del modelo, comparando los resultados entre los clasificadores KNN y regresión logística. Se espera que los estudiantes generen visualizaciones que evidencien las diferencias en la importancia de las características y discutan la coherencia de los resultados con el problema planteado.

Se presenta un ejemplo del uso de SHAP aplicado a un conjunto de datos distinto, pero con el mismo objetivo: entrenar modelos de clasificación utilizando *K-Nearest Neighbors* (KNN) y *Logistic Regression* (LR), y posteriormente aplicar SHAP para analizar la explicabilidad de las predicciones de ambos modelos.

```

1 import shap
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import load_breast_cancer
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.linear_model import LogisticRegression
7 from sklearn.neighbors import KNeighborsClassifier
8
9 # --- Cargar y preparar los datos ---
10 X, y = load_breast_cancer(return_X_y=True)
11 feature_names = load_breast_cancer().feature_names
12 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
13                                                    random_state=42)
14 scaler = StandardScaler()
15 X_train_std = scaler.fit_transform(X_train)
16 X_test_std = scaler.transform(X_test)
17
18 # --- Entrenar los modelos ---
19 knn = KNeighborsClassifier(n_neighbors=5)
20 lr = LogisticRegression(max_iter=1000)
21 knn.fit(X_train_std, y_train)
22 lr.fit(X_train_std, y_train)
23
24 # --- Explicabilidad con SHAP ---
25 # Para regresión logística (modelo lineal)
26 explainer_lr = shap.LinearExplainer(lr, X_train_std)
27 shap_values_lr = explainer_lr(X_test_std)

```

```
27
28 # Para KNN (modelo no lineal)
29 explainer_knn = shap.KernelExplainer(knn.predict_proba , X_train_std[:100])
30 shap_values_knn = explainer_knn.shap_values(X_test_std[:1]) # una prediccion
```

Para más información sobre el uso de SHAP en explicaciones locales y globales consulte: Documentación oficial de SHAP.

Referencias

- [1] Andras Janosi et al. “Heart disease. uci machine learning repository”. En: *UCI Machine Learning Repository* (1988).
- [2] Gareth James et al. *An introduction to statistical learning: With applications in python*. Springer Nature, 2023.